

PWU-AB-2 RAUC dm-verity A/B update

PWU-AB-2 — RAUC dm-verity A/B update: env-scheme reconciliation + apply→verify→mark-good→rollback design

Revision: 1 Last modified: 2026-06-11T00:00:00Z Status (§11.4.6): DESIGN + CONFIG ARTIFACTS ONLY — nothing here is a proven RAUC update. The config files in `../../../../tests/emulator/ab_virt/rauc/` are authored, parse-clean, and internally coherent with the proven GPT layout; **no RAUC bundle has been built and no rauc install has run**. Every “MUST build” / “does not exist yet” / “to be run by the conductor” marker below is honest. The two proven pieces this builds on are PWU-AB-1 (slot switch) and PWU-AB-3 (auto-rollback), both verified live on macOS under real **U-Boot 2024.01 + QEMU -machine virt + HVF** (evidence: `uboot_ab/README.md` Status §; docs/qa/20260611T094958Z-ab-slot-switch/, docs/qa/20260611T095918Z-ab-rollback/).

Authoritative parent design: `../../../../research/rk3588_emulator/REPORT.md` (PWU-AB-2). State machine this reconciles with: `../../../../tests/emulator/ab_virt/uboot_ab/README.md` + `../../../../tests/emulator/ab_virt/uboot_ab/boot.cmd`. Test driver this unblocks: `../../../../tests/emulator/ab_virt/ab_rauc_verity.sh` (its TODO a–d). Shared fw_env dependency: `PWU_AB_4_APPLY_PORT.md` §3.

1. The integration gap (the thing this PWU exists to close)

`ab_rauc_verity.sh`’s §11.4.6 honest gap is real: RAUC’s native U-Boot backend and this project’s `boot.cmd` express A/B slot state in **two different counter schemes** that share ONE anchor variable (`BOOT_ORDER`). The reconciliation must let a single `boot.scr` serve BOTH the proven manual bootcount path (PWU-AB-1/AB-3) AND a real `rauc install` apply, without breaking either.

1.1 RAUC's U-Boot env scheme (from the upstream sources, verbatim §7)

RAUC's [system] bootloader=uboot backend (src/bootchooser.c, the contrib/uboot.sh example boot script, and the upstream U-Boot bootmeth_rauc) uses:

Variable	Meaning	Who writes it
BOOT_ORDER	space-separated bootname list; head = slot tried first ("A B" ⇒ try A then B).	rauc (mark-active / install) + the boot script (on exhaustion-reset). the boot script decrements the chosen slot by 1 each boot; rauc status mark-good <slot> resets the booted slot's LEFT to its default ; rauc status mark-bad sets it to 0.
BOOT_<bootname>_LEFT	per-slot remaining boot attempts (BOOT_A_LEFT, BOOT_B_LEFT). Initialised to 3 (the upstream default / CONFIG_BOOTMETH_RAUC_DEFAULT_TRIES).	the boot script appends it to bootargs for the chosen slot.
rauc.slot (kernel arg)	tells the booted userspace which bootname it is, so rauc can self-detect the active slot.	

RAUC's boot algorithm (the contrib/uboot.sh example, verbatim in §7): iterate BOOT_ORDER; for the first slot whose BOOT_<bootname>_LEFT > 0, **decrement that slot's LEFT, set bootargs (root + rauc.slot=<name>), saveenv, and boot it**. If NO slot has attempts left, reset all LEFT counters to default and reset (reboot). The “promotion” of a slot is BOOT_ORDER head-order; the “demotion” of a failing slot is its LEFT reaching 0 (so the iteration skips it and the next-in-order slot wins).

1.2 This project's `boot.cmd` scheme (proven, PWU-AB-1/AB-3)

Variable	Meaning
<code>BOOT_ORDER</code>	head = active slot — IDENTICAL semantics to RAUC. This is the shared anchor.
<code>bootcount / bootlimit</code>	a SINGLE global attempt counter (U-Boot's <code>CONFIG_BOOTCOUNT_LIMIT</code> increments <code>bootcount</code> before the script runs; the script's guard swaps <code>BOOT_ORDER</code> when <code>bootcount > bootlimit</code>).
<code>upgrade_available</code>	1 = update on probation (counter armed); 0 = confirmed-good (counter frozen). Gates whether <code>bootcount</code> is saved/incremented.
<code>helix_slot</code> (kernel arg)	this project's analogue of <code>rauc.slot</code> — already on <code>/proc/cmdline</code> (<code>boot.cmd:100</code>).

1.3 Where they agree and disagree

- **Agree (load-bearing):** `BOOT_ORDER` head = active slot. PWU-AB-1/AB-3 already adopted the RAUC/Mender `BOOT_ORDER` convention — that is why this reconciliation is *additive*, not a rewrite.
- **Disagree:** the counter. RAUC = **per-slot** `BOOT_<bootname>_LEFT` (multi-try, slot-local demotion). This project = **global** `bootcount/bootlimit` + **upgrade_available gate** (single-try probation, swap-on-exhaust). They are NOT the same variable and a naive “use both” would let two independent rollback engines fight over `BOOT_ORDER`.

2. Reconciliation DECISION

DECISION: EXTEND `boot.cmd` to ALSO honor RAUC's per-slot `BOOT_<bootname>_LEFT` counters as an ADDITIVE second layer, keeping the proven `bootcount/bootlimit/upgrade_available` guard byte-for-byte intact. Do NOT replace `boot.cmd` with the stock `contrib/uboot.sh`.

2.1 Why extend-our-`boot.cmd`, NOT adopt `rauc's uboot.sh`

1. **Preserves the GREEN proofs (the overriding constraint, §11.4.114).** PWU-AB-1 (slot switch) and PWU-AB-3 (auto-rollback) are GREEN against the *current* `boot.cmd` on the *real* `u-boot.bin` artifact. Replacing it with `contrib/uboot.sh` would discard the proven `bootcount/bootlimit/upgrade_available` guard those tests assert on — a regression of two passing PWUs to win one. The constitution's known-good-tag discipline (§11.4.114) makes “keep the proven mechanism, add to it” the correct path.

2. **contrib/uboot.sh is explicitly an EXAMPLE, target-specific.** Its verbatim body (§7) hard-codes `nand read ... ${kernel_a_nandoffset}` for kernel-on-NAND + rootfs-on-eMMC (`/dev/mmcblk0p1`) — neither matches this emulator (kernel Image on a FAT partition loaded via `load virtio 0:1`, rootfs on `/dev/vda2//dev/vda3`). RAUC’s own docs say “you’ll have to adjust it for your use-case” and “for minor changes in boot logic or variable names simply change the boot script and/or the RAUC `system.conf` bootname settings.” Adapting it = re-deriving `boot.cmd`’s load path anyway, with the proofs thrown away.
3. **The anchor already matches.** Both schemes pick the active slot by the **head of BOOT_ORDER**. `rauc status mark-active <slot>` (and `rauc install`’s activation) **write BOOT_ORDER with the new slot at the head** — exactly what `boot.cmd`’s `setexpr active_slot sub "[ ].*${}" "" "${BOOT_ORDER}"` already reads. So a `rauc install` that re-orders `BOOT_ORDER` to `head=B` is *already honored* by the proven head-select logic with ZERO change. The slot-switch half of the RAUC apply works against the unmodified `boot.cmd`.
4. **Additive BOOT_<bootname>_LEFT honoring is a small, safe superset.** The only new behaviour needed is: when RAUC has armed per-slot LEFT counters, `boot.cmd` should (a) skip a head slot whose `BOOT_<head>_LEFT == 0` (RAUC demotion), and (b) decrement the chosen slot’s `LEFT + saveenv` (so `rauc status mark-good` — which resets `LEFT` — closes the probation). This is layered AROUND the existing guard, default-inert when the `LEFT` vars are absent (the PWU-AB-1/AB-3 path, which sets no `LEFT` vars, is unchanged).

2.2 The extended boot.cmd state machine (DESIGN — to be implemented + re-proven)

This is the **design** for the `boot.cmd` extension. It is NOT yet written into `boot.cmd` (that is a separate PWU step with its own §11.4.115 RED→GREEN + independent review §11.4.142). The current `boot.cmd` is unchanged and its PWU-AB-1/AB-3 proofs stand.

Insertion point: AFTER the existing `bootcount > bootlimit` guard (`boot.cmd:59–69`) and BEFORE the head-slot `setexpr` (`boot.cmd:75`). The existing guard and head-select stay verbatim. New additive block (hush, mirrors `contrib/uboot.sh` iteration but adapted to this emulator’s `load virtio + /dev/vda` layout):

```
# ---- RAUC per-slot attempt layer (ADDITIVE; inert when LEFT vars
absent) ----
# Only engages when a `rauc install`/mark-active armed per-slot
counters. When
# absent (the PWU-AB-1/AB-3 manual-bootcount path) this whole
block is a no-op
# because the `test -n` guards leave the chosen slot as
BOOT_ORDER's head.
setenv rauc_chosen ""
for s in ${BOOT_ORDER}; do
    if test -z "${rauc_chosen}"; then
        # read this slot's LEFT counter (BOOT_A_LEFT / BOOT_B_LEFT);
absent => treat
        # as "not RAUC-managed" so the head wins (preserves PWU-AB-1/
AB-3).
        if test "${s}" = "A"; then setenv _left "${BOOT_A_LEFT}"; else
```

```

setenv _left "${BOOT_B_LEFT}"; fi
    if test -n "${_left}"; then
        if test 0x${_left} -gt 0; then
            echo "A/B(RAUC): slot ${s} has ${_left} attempts left ->
choosing it"
            setenv rauc_chosen "${s}"
            # decrement + persist so a never-confirmed slot exhausts
its tries and
            # the NEXT boot's iteration skips it (RAUC demotion).
            if test "${s}" = "A"; then setexpr BOOT_A_LEFT $
{BOOT_A_LEFT} - 1; else setexpr BOOT_B_LEFT ${BOOT_B_LEFT} - 1; fi
            saveenv
        else
            echo "A/B(RAUC): slot ${s} exhausted (LEFT=0) -> skip"
        fi
    else
        # LEFT var absent for this slot => non-RAUC-managed; head
wins.
        setenv rauc_chosen "${s}"
    fi
fi
done
# If RAUC counters were present and ALL exhausted, rauc_chosen is
"" -> reset to
# default tries and reboot (mirrors contrib/uboot.sh "No valid
slot" recovery).
if test -n "${BOOT_A_LEFT}${BOOT_B_LEFT}"; then
    if test -z "${rauc_chosen}"; then
        echo "A/B(RAUC): no slot with attempts left -> reset tries to
3 + reboot"
        setenv BOOT_A_LEFT 3
        setenv BOOT_B_LEFT 3
        saveenv
        reset
    fi
    # Promote the RAUC-chosen slot to the head so the existing
setexpr picks it.
    if test "${rauc_chosen}" = "A"; then setenv BOOT_ORDER "A B";
else setenv BOOT_ORDER "B A"; fi
fi

```

The existing setexpr active_slot sub "[].*\$" "" "\${BOOT_ORDER}" ([boot.cmd:75](#)) then reads the (now possibly RAUC-promoted) head, and the existing root_part map + load virtio 0:1 ... Image + booti run UNCHANGED. The kernel bootargs line gains rauc.slot=\${active_slot} alongside the existing helix_slot=\${active_slot} so in-guest rauc self-detects the slot (system.conf bootname matches).

Coexistence rule (the one invariant that keeps both engines from fighting): the global bootcount/bootlimit guard fires ONLY when upgrade_available != 0 causes U-Boot to increment bootcount past bootlimit. A RAUC apply that arms per-slot LEFT counters SHOULD leave upgrade_available=0 (so the global guard stays inert) and rely on the LEFT layer for demotion — OR set upgrade_available=1 and use the global guard, but NOT both for the same apply.

The two paths are mutually exclusive per apply; `boot.cmd`'s `LEFT` block is inert when `LEFT` vars are absent, and the global guard is inert when `upgrade_available=0`. PWU-AB-1/AB-3 use the global path (no `LEFT` vars); a `rauc install` uses the `LEFT` path (`upgrade_available` untouched at 0).

2.3 Honest boundary (§11.4.6)

- The §2.2 extension is **designed, not implemented or proven**. Its hush syntax (for `s` in `${BOOT_ORDER}`, nested `if test ... -gt 0, setexpr ... - 1`) mirrors the verbatim `contrib/uboot.sh` (§7) which is known-good U-Boot hush, but it has NOT been compiled via `mkimage` or run under U-Boot in THIS emulator. A separate PWU implements + re-proves it (§11.4.115
RED→GREEN: assert the current `boot.cmd` does NOT honor `LEFT` counters → wire the block → assert it does, both PWU-AB-1/AB-3 still GREEN).
- The simplest *first* GREEN for PWU-AB-2 does NOT even need §2.2: because `rauc install/mark-active` re-orders `BOOT_ORDER` to `head=new-slot`, and the proven `boot.cmd` already selects the head, a single-try RAUC apply (`LEFT` default 1, or relying purely on the head re-order + the existing global guard for rollback) switches slots against the UNMODIFIED `boot.cmd`. The §2.2 multi-try `LEFT` layer is the *full-fidelity* RAUC behaviour; the *minimal* PWU-AB-2 GREEN can ride the existing head-select. The test plan (§6) proves the minimal path first.

3. The apply → verify → mark-good → rollback design (mapped to rauc commands + the GPT layout)

GPT layout (the load-bearing contract, UNCHANGED from `assemble_ab_disk.sh` + `boot.cmd:34-40`):

Part	Partlabel	Role	RAUC bootname	Device
p1	boot	FAT: kernel Image + <code>boot.scr</code> (+ env, PWU-AB-4 §3)	—	load virtio 0:1
p2	rootfs_a	slot A root	A	/dev/vda2
p3	rootfs_b	slot B root	B	/dev/vda3

Phase	rauc command (in-guest)	Effect on GPT + env	Runtime signature (§11.4.108)
baseline	<code>rauc status --detailed</code>	none	reports slot A booted+good, B inactive
apply	<code>rauc install /</code>	verifies bundle sig against keyring → writes	<code>rauc install rc=0; /dev/</code>

Phase	rauc command (in-guest)	Effect on GPT + env	Runtime signature (§11.4.108)
activate	root/ update.raucb	the verity rootfs image to the inactive slot (B = /dev/vda3) → activates B: writes BOOT_ORDER="B A" (+ arms BOOT_B_LEFT to default via the U-Boot backend / fw_setenv)	vda3 block hash changed (real bytes written)
	(done by rauc install's U-Boot backend, OR explicitly rauc status mark-active other)	BOOT_ORDER head = B; per-slot LEFT armed	fw_printenv BOOT_ORDER ⇒ B A
	reboot	U-Boot re-sources boot.scr → head=B → root=/dev/vda3 rauc.slot=B helix_slot=B	/proc/cmdline has helix_slot=B; findmnt / ⇒ verity dm device over /dev/vda3
verify (verity)	(kernel brings up dm-verity for the verity-mounted root)	dm-verity target active over the slot device	dmsetup status ⇒ ≥1 verity target; dmesg ⇒ device-mapper: verity
mark-good	rauc status mark-good booted	resets the booted slot's BOOT_<bootname>_LEFT to default ⇒ slot CONFIRMED, no rollback	fw_printenv BOOT_B_LEFT ⇒ default (e.g. 3); rauc status ⇒ B good
rollback (unhealthy)	(mark-good NEVER runs)	each boot decrements BOOT_B_LEFT; once 0 the §2.2 iteration skips B → head A wins → boots /dev/vda2	post-rollback helix_slot=A / etc/slot_id=A — composes proven PWU-AB-3

Verity-specific note (§11.4.6 honest): format=verity makes the *bundle* integrity-protected (the bundle's payload is a dm-verity image whose root hash is in the signed manifest; RAUC verifies the whole bundle's signature + the verity hash before install). Whether the *installed slot* is then mounted **as** a dm-verity device at runtime depends on how the rootfs image + the slot are configured (a verity-protected rootfs needs the verity metadata on/beside the slot + the kernel/initramfs to set up the dm-verity target with the manifest's root hash). The minimal PWU-AB-2 GREEN proves the **apply+slot-switch** under real RAUC; the **runtime dm-verity-active root** is the stronger signature (HELIX_DMVERITY=[1-9] in

ab_rauc_verity.sh) and may require an initramfs/verity-mount step that is called out as a follow-on if the plain ext4 slot mount does not bring up a verity target. Do NOT claim runtime verity until dmsetup status shows it.

4. Concrete bundle-build recipe (the conductor's container step — NOT run here)

All of this runs in the aarch64 build container (like build_image.sh), writing into the **gitignored** out/ tree. **No key is committed (§11.4.10)**. The keygen helper (../.../tests/emulator/ab_virt/rauc/gen_dev_keys.sh) generates a throwaway self-signed dev cert+key into out/rauc-keys/ at build time.

4.1 Dev signing key + cert (self-signed, throwaway — §11.4.10)

```
# (gen_dev_keys.sh does exactly this, into out/rauc-keys/, key
    chmod 600)
openssl req -x509 -newkey rsa:4096 -nodes \
    -keyout out/rauc-keys/dev.key.pem \
    -out out/rauc-keys/dev.cert.pem \
    -subj "/O=Helix OTA dev/CN=helix-ota-ab-virt-dev" -days 365
```

The SAME dev.cert.pem is the bundle's signing cert AND the guest keyring (/etc/rauc/system.conf [keyring] path=) — self-signed dev trust. A production build uses a real PKI (§11.4.10); this is a dev-only emulator key, never committed, never reused.

4.2 Manifest (format=verity)

Template: ../.../tests/emulator/ab_virt/rauc/manifest.raucm.in. The build substitutes @COMPATIBLE@, @VERSION@, @ROOTFS_IMAGE@:

```
[update]
compatible=helix-ota-ab-virt
version=@VERSION@
```

```
[bundle]
format=verity
```

```
[image.rootfs]
filename=@ROOTFS_IMAGE@
# sha256 + size are filled in automatically by `rauc bundle`.
```

4.3 Build the bundle

```
# stage the new rootfs image (the slot payload) into a temp bundle
    dir, render
# the manifest, then sign+pack. RAUC computes the verity hash +
    the image
# sha256/size automatically for format=verity.
```



```

mkdir -p out/bundle-src
cp out/images/rootfs.ext2 out/bundle-src/rootfs.ext4.img # the slot payload
sed -e "s/@COMPATIBLE@/helix-ota-ab-virt/" \
    -e "s/@VERSION@/$(date -u +%Y.%m.%d)-1/" \
    -e "s/@ROOTFS_IMAGE@/rootfs.ext4.img/" \
    tests/emulator/ab_virt/rauc/manifest.raucm.in > out/bundle-
src/manifest.raucm
rauc --cert out/rauc-keys/dev.cert.pem --key out/rauc-keys/
dev.key.pem \
    bundle out/bundle-src/ out/images/update.raucb

```

Then stage out/images/update.raucb into the guest at RAUC_BUNDLE (default / root/update.raucb) — e.g. via a second boot partition file, a 9p/virtfs mount, or by writing it into a slot the test reads. (The exact staging path is the conductor's choice at run time; ab_rauc_verity.sh reads RAUC_BUNDLE.)

4.4 /etc/rauc/system.conf (in BOTH slot rootfs)

Authored: ../../../../tests/emulator/ab_virt/rauc/system.conf. Bound to THIS emulator's layout:

```

[system]
compatible=helix-ota-ab-virt
bootloader=uboot
mountprefix=/mnt/rauc
# bundle-formats: allow verity (the bundle we build); -plain forbids unsigned plain.
bundle-formats=-plain

[keyring]
path=/etc/rauc/dev.cert.pem

[slot.rootfs.0]
device=/dev/vda2
type=ext4
bootname=A

[slot.rootfs.1]
device=/dev/vda3
type=ext4
bootname=B

```

bootname=A/bootname=B match the BOOT_ORDER tokens boot.cmd already reads (§2). device=/dev/vda2//dev/vda3 match the proven GPT contract. bootloader=uboot selects RAUC's U-Boot backend, which reads/writes BOOT_ORDER + BOOT_<bootname>_LEFT via fw_setenv/fw_printenv (§5).

4.5 /etc/fw_env.config (in BOTH slot rootfs — SHARED with PWU-AB-4 §3)

Authored: ../../../../tests/emulator/ab_virt/rauc/fw_env.config. **This is the single biggest UNVERIFIED dependency** (§11.4.6): it **MUST** point fw_setenv/fw_printenv at the EXACT location the QEMU u-boot.bin stores its environment (CONFIG_ENV_IS_IN_*). For the qemu_arm64 U-Boot defconfig that is **most commonly CONFIG_ENV_IS_IN_FAT** (env in a file uboot.env on the FAT boot partition) — but it **MAY** be a raw offset (CONFIG_ENV_IS_IN_* raw) depending on the built config. The authored file documents BOTH candidate forms and is marked UNVERIFIED until checked against the built u-boot.bin:

```
# CANDIDATE A (CONFIG_ENV_IS_IN_FAT – env as a file on the FAT
boot partition):
#   fw_env.config does NOT support FAT-file env directly; if the
built U-Boot
#   uses ENV_IS_IN_FAT, the persistent-env work in PWU_AB_4 §3
must instead
#   place a raw env region OR use libubootenv's FAT support where
available.
# CANDIDATE B (raw env region on the boot device – RECOMMENDED to
wire so
#   fw_setenv works): a single env copy at a fixed offset on /dev/
vda1.
#   <device> <offset> <env-size> <sector-size>
# /dev/vda1 0x000000 0x4000 0x200
#
# UNVERIFIED (§11.4.6): the real offset/size/redundancy MUST be
read from the
# built u-boot.bin CONFIG_ENV_* before this is trusted. See
PWU_AB_4 §3 +
# the Negative finding in §8.
```

§11.4.6 / §11.4.108 warning: if fw_env.config does not match the bootloader's env storage byte-for-byte, the guest writes an env the bootloader never reads — a silent SOURCE→RUNTIME gap. The conductor **MUST** verify fw_printenv round-trips against U-Boot's view (set in U-Boot, read with fw_printenv; set with fw_setenv, read at the => prompt) **BEFORE** trusting any rauc install slot arm. This verification is a gating step of the bundle-build PWU.

4.6 Guest package dependency

u-boot-tools (fw_setenv/fw_printenv, libubootenv) **MUST** be in the guest. build_image.sh currently has BR2_PACKAGE_RAUC=y + the U-Boot target but **not** an explicit BR2_PACKAGE_UBOOT_TOOLS=y for the *target* — that add is part of the bundle-build PWU (a build_image.sh change, separate commit + review §11.4.142). ab_rauc_verity.sh already calls fw_setenv (line 237), so the test will surface its absence honestly as a non-zero rc (the RED baseline).

5. How RAUC drives the env (the wire between rauc and boot.cmd)

1. `rauc install` (or `rauc status mark-active other`) → RAUC's U-Boot backend calls `fw_setenv` to write `BOOT_ORDER` with the new slot at the head AND set the new slot's `BOOT_<bootname>_LEFT` to the default tries.
2. Reboot → `boot.scr` (extended `boot.cmd`, §2.2) reads `BOOT_ORDER` head + (additively) the `BOOT_<bootname>_LEFT` counter, decrements it, `saveenv`, boots the slot with `rauc.slot=<name>`.
3. In-guest `rauc status mark-good booted` → `fw_setenv` resets the booted slot's `BOOT_<bootname>_LEFT` to default ⇒ slot confirmed (probation closed). This is the RAUC analogue of PWU-AB-4 §4's healthy-marker (`fw_setenv upgrade_available 0`; `bootcount 0`); per §11.4.74 the RAUC bundle path SHOULD reuse `rauc status mark-good` as the marker.
4. Unhealthy: `mark-good` never runs → `BOOT_<bootname>_LEFT` decrements each boot → reaches 0 → §2.2 iteration skips it → head falls through to the other slot ⇒ auto-rollback (composes proven PWU-AB-3).

All of (1)/(3) require the §4.5 `fw_env.config` to be correct — that is the gating verification.

6. RED → GREEN test plan (references `ab_rauc_verity.sh`)

The driver `ab_rauc_verity.sh` is authored with `RED_MODE` (default 1) per §11.4.115. This plan maps its assertions to the build order.

6.1 RED baseline (`RED_MODE=1`, current artifact — NO bundle, NO `system.conf` wired)

Run AS-IS once PWU-AB-1 is GREEN and the disk exists. Expected: `rauc install` returns non-zero (no bundle / no `system.conf`), no dm-verity target, slot stays A. The driver's RED assertions (`ab_rauc_verity.sh:327-330`) assert exactly this defect-present baseline → captures the honest “apply unproven” state. A GREEN here would be a §11.4 bluff.

6.2 Wire the minimal apply (rides the UNMODIFIED `boot.cmd` head-select, §2.3)

1. Build the bundle (§4.3) + stage into the guest.
2. Add `system.conf` (§4.4) + `fw_env.config` (§4.5, **verified** §4.5 warning) + `u-boot-tools` (§4.6) to BOTH slots via `build_image.sh` overlay; verify `fw_printenv/fw_setenv` round-trip against U-Boot (the gating check).
3. `rauc install` writes B + re-orders `BOOT_ORDER="B A"` (head=B). The proven `boot.cmd` head-select already boots B. Flip `RED_MODE=0`.

6.3 GREEN guard (RED_MODE=0)

The SAME driver assertions (ab_rauc_verity.sh:319–323) now guard the GREEN behaviour: HELIX_RAUC_INSTALL_RC=0, HELIX_POSTSLOT=B (slot switched by the real RAUC apply), HELIX_ROOTDEV= captured, HELIX_DMVERITY=[1–9] (verity-active — see the §3 verity honesty note; if runtime verity needs the initramfs step, that is a flagged follow-on and this assertion stays RED until it lands — do NOT force GREEN), and not HELIX_POSTSLOT=A.

6.4 Full coverage (§11.4.27 / §11.4.85 / §1.1)

- **Healthy path:** install B → reboot → rauc status mark-good booted → B confirmed (BOOT_B_LEFT reset). Evidence: fw_printenv before/after mark-good.
- **Unhealthy path (composes PWU-AB-3):** install B → reboot → mark-good never runs → BOOT_B_LEFT decrements to 0 → §2.2 skip → rollback to A. Asserts HELIX_POSTSLOT=A after the exhaustion reboot.
- **Chaos (§11.4.85):** power-cut (QMP) during rauc install slot write → the inactive slot is either old-consistent or the install is re-runnable (RAUC writes the inactive slot, never the running one — no brick); power-cut during the fw_setenv arm → env is old-or-new, never half (the §4.5 verification proves atomicity).
- **Determinism (§11.4.50):** N identical PASS + identical evidence hashes on the same disk MD5 + bundle MD5.
- **§1.1 paired mutation:** (a) corrupt the bundle signature → rauc install MUST FAIL (proves the keyring gate is real, not bypassed); (b) point system.conf [slot.rootfs.1] device= at /dev/vda2 (the ACTIVE slot) → RAUC MUST refuse to install onto the booted slot (proves slot-detection is real).

6.5 Evidence (§11.4.5/§11.4.69/§11.4.83/§11.4.107)

Under docs/qa/<run-id>-ab-rauc-verity/ (the driver already writes these): console.log (full U-Boot+kernel+guest, live not frozen — the HELIX_DONE_RAUC_MARK sentinel proves a live post-reboot shell), rauc_status_pre.txt/rauc_status_post.txt, verdict.txt. Add fw_env_before/after.txt (the env transition = the §11.4.108 runtime signature) + slot_block_hash.txt (pre/post inactive-slot hash = real bytes written, not faked) when the apply is wired.

7. Verbatim upstream RAUC contrib/uboot.sh (the reference algorithm, §11.4.99)

Fetches 2026-06-11 from <https://raw.githubusercontent.com/rauc/rauc/master/contrib/uboot.sh>:

```
# This is only an example to show a possible bootcmd script to
    implement A/B
# slot selection for kernel on NAND and rootfs on eMMC. It's
    expected that
```

```

# you'll have to adjust it for your use-case.

test -n "${BOOT_ORDER}" || setenv BOOT_ORDER "A B"
test -n "${BOOT_A_LEFT}" || setenv BOOT_A_LEFT 3
test -n "${BOOT_B_LEFT}" || setenv BOOT_B_LEFT 3

setenv bootargs
for BOOT_SLOT in "${BOOT_ORDER}"; do
    if test "x${bootargs}" != "x"; then
        # skip remaining slots
    elif test "x${BOOT_SLOT}" = "xA"; then
        if test 0x${BOOT_A_LEFT} -gt 0; then
            echo "Found valid slot A, ${BOOT_A_LEFT} attempts remaining"
            setexpr BOOT_A_LEFT ${BOOT_A_LEFT} - 1
            setenv load_kernel "nand read ${kernel_loadaddr} $
                {kernel_a_nandoffset} ${kernel_size}"
            setenv bootargs "${default_bootargs} root=/dev/mmcblk0p1
                rauc.slot=A"
        fi
    elif test "x${BOOT_SLOT}" = "xB"; then
        if test 0x${BOOT_B_LEFT} -gt 0; then
            echo "Found valid slot B, ${BOOT_B_LEFT} attempts remaining"
            setexpr BOOT_B_LEFT ${BOOT_B_LEFT} - 1
            setenv load_kernel "nand read ${kernel_loadaddr} $
                {kernel_b_nandoffset} ${kernel_size}"
            setenv bootargs "${default_bootargs} root=/dev/mmcblk0p2
                rauc.slot=B"
        fi
    fi
done

if test -n "${bootargs}"; then
    saveenv
else
    echo "No valid slot found, resetting tries to 3"
    setenv BOOT_A_LEFT 3
    setenv BOOT_B_LEFT 3
    saveenv
    reset
fi

echo "Loading kernel"
run load_kernel
echo " Starting kernel"
bootm ${loadaddr_kernel}

```

This is the EXAMPLE this project ADAPTS (not adopts): the nand read ... mmcblk0p1 load/root lines are replaced by this emulator's load virtio 0:1 ... Image + root=/dev/vda\${root_part} (the §2.2 block keeps the BOOT_ORDER/ BOOT_<bootname>_LEFT iteration + decrement + reset-and-reboot recovery, drops the NAND/mmc specifics).

8. Sources verified 2026-06-11

- **RAUC U-Boot integration** ([system] bootloader=u-boot, [slot.rootfs.N] bootname/device, BOOT_ORDER + BOOT_<bootname>_LEFT, fw_setenv/fw_printenv + /etc/fw_env.config, contrib/u-boot.sh): <https://rauc.readthedocs.io/en/latest/integration.html>
- **RAUC examples** (rauc bundle args, openssl req -x509 -newkey rsa:4096 dev cert, manifest.raucm [bundle] format=verity + [image.rootfs], system.conf [keyring], rauc install): <https://rauc.readthedocs.io/en/latest/examples.html>
- **RAUC contrib/u-boot.sh** (verbatim algorithm, §7): <https://raw.githubusercontent.com/rauc/rauc/master/contrib/u-boot.sh>
- **U-Boot RAUC Bootmeth** (BOOT_ORDER, BOOT_<bootname>_LEFT, CONFIG_BOOTMETH_RAUC_DEFAULT_TRIES, decrement-by-1, all-zero reset, raucargs/rauc.slot): <https://docs.u-boot.org/en/latest/develop/bootstd/rauc.html>
- **U-Boot Boot Count Limit** (this project's bootcount/bootlimit/altbootcmd/upgrade_available scheme, the global-counter layer §1.2): <https://docs.u-boot.org/en/latest/api/bootcount.html>
- **U-Boot environment tools** (fw_setenv/fw_printenv, fw_env.config): <https://docs.u-boot.org/en/latest/develop/environment.html>
- **Mender U-Boot integration** (BOOT_ORDER head = active slot, the shared anchor): <https://docs.mender.io/operating-system-updates-yocto-project/board-integration/bootloader-support/u-boot/manual-u-boot-integration>

Negative finding (§11.4.99): RAUC's official full-system QEMU example is **x86 + GRUB**, NOT arm64 + U-Boot — the GRUB env-block mechanics do NOT transfer, and the arm64+U-Boot fw_env.config offset/size is target-specific and MUST be read from THIS emulator's built u-boot.bin CONFIG_ENV_IS_IN_* (the qemu_arm64 defconfig commonly uses CONFIG_ENV_IS_IN_FAT, which libubootenv's classic fw_env.config raw-offset form does NOT directly support — this is the §4.5 UNVERIFIED gating dependency, shared with PWU_AB_4_APPLY_PORT.md §3). Do NOT assume the upstream example's offsets transfer.

Negative finding (§11.4.6 — runtime verity): [bundle] format=verity protects the BUNDLE (signed manifest + verity-hashed payload, verified before install). It does NOT by itself guarantee the INSTALLED slot is mounted as a runtime dm-verity device — that needs the verity metadata available to the booted kernel/initramfs + a dm-verity mount step. The HELIX_DMVERITY=[1-9] assertion in ab_rauc_verity.sh is the runtime-verity signature and stays RED until that mount path is proven; the minimal PWU-AB-2 GREEN proves apply+slot-switch under real RAUC, not necessarily runtime-verity. No runtime-verity claim is made until dmsetup status shows it.